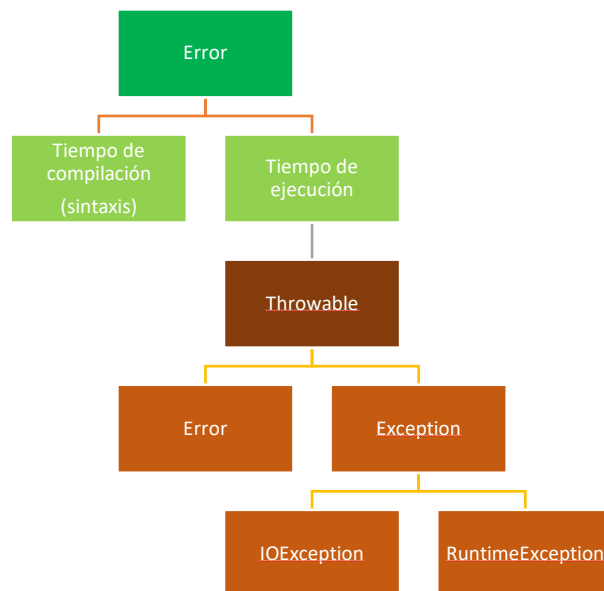


Excepciones en Java

Introducción a las excepciones

Las excepciones son eventos inesperados que pueden ocurrir durante la ejecución de un programa. En Java, **las excepciones son objetos** que representan errores o condiciones anormales que surgen durante la ejecución de un programa. Estos eventos pueden ser causados por diversos factores, como errores en el código, falta de recursos, entrada de datos incorrecta, entre otros.

El manejo de excepciones en Java es fundamental para garantizar la robustez, la estabilidad y la seguridad de un programa. El objetivo de este capítulo es explicar cómo se pueden manejar las excepciones en Java para evitar interrupciones inesperadas y proporcionar una experiencia de usuario adecuada.



Clases de excepciones

En Java, las excepciones se clasifican en dos categorías principales:

1. **Excepciones comprobadas (checked exceptions):** Estas excepciones heredan de la clase `IOException`. Son excepciones que deben ser manejadas explícitamente en el código, utilizando bloques try-catch o declarando la excepción en la firma del método con la palabra clave "throws". Algunas excepciones comprobadas comunes son `FileNotFoundException`, `IOException`, `SQLException`, etc.
2. **Excepciones no comprobadas (unchecked exceptions):** Estas excepciones heredan de la clase `RuntimeException` y no es obligatorio manejarlas en el código. Sin embargo, es recomendable hacerlo para evitar comportamientos inesperados. Algunas excepciones no comprobadas comunes son `NullPointerException`, `IndexOutOfBoundsException`, `ArithmeticException`, etc.

IOException
(excepciones comprobadas)



Programador Java

RuntimeException
(excepciones no comprobadas)



Programador Java



Manejo de excepciones

El manejo de excepciones en Java se realiza utilizando bloques try, catch y finally, así como la declaración "throws". A continuación, se describen estos elementos:

1. Bloque try: Este bloque contiene el código que puede generar una excepción. Si se produce una excepción dentro de este bloque, se pasa al bloque catch correspondiente.

```
try {  
    // Código que puede generar una excepción  
}
```

2. Bloque catch: Este bloque maneja la excepción generada en el bloque try. Puede haber múltiples bloques catch para manejar diferentes tipos de excepciones. Los bloques catch deben seguir inmediatamente al bloque try.

```
catch (ExceptionType1 e) {  
    // Código para manejar la excepción de tipo ExceptionType1  
}  
catch (ExceptionType2 e) {  
    // Código para manejar la excepción de tipo ExceptionType2  
}
```

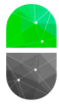
3. Bloque finally: Este bloque contiene el código que se ejecutará siempre, independientemente de si se produjo una excepción o no. El bloque finally es opcional y se coloca después de los bloques catch.

```
finally {  
    // Código que se ejecutará siempre  
}
```

4. Palabra clave "throws": Se utiliza para indicar que un método puede lanzar una excepción comprobada. La excepción debe ser manejada por el método que llama a este método.

```
public void methodName() throws ExceptionType {  
    // Código del método  
}
```





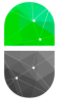
Ejemplo sencillo con bloque try-catch: Este ejemplo divide dos números ingresados por el usuario. Si el divisor es cero, se generará una **ArithmeticException** y se manejará dentro del bloque **catch**.

```
5 import java.util.Scanner;
6
7 public class DivisionTryCatch {
8
9     public static void main(String[] args) {
10
11         Scanner scanner = new Scanner(System.in);
12
13         System.out.println("Introduce el dividendo:");
14
15         int dividendo = scanner.nextInt();
16
17         System.out.println("Introduce el divisor:");
18
19         int divisor = scanner.nextInt();
20
21         try {
22
23             int resultado = dividendo / divisor;
24
25             System.out.println("El resultado de la división es: " + resultado);
26
27         } catch (ArithmeticException e) {
28
29             System.out.println("Error: División por cero no permitida.");
30
31         }
32     }
33 }
```

En este ejemplo, si el divisor es cero, el bloque **try** lanzará una **ArithmeticException**, y el bloque **catch** capturará la excepción y mostrará un mensaje de error. Si no se produce ninguna excepción, el programa simplemente mostrará el resultado de la división.

Se puede agregar adicionalmente un bloque **finally** (no obligatorio) que se ejecute independientemente de si se produce o no una excepción. En este caso además sería conveniente para asegurarnos de que ocurra lo que ocurra, el scanner se cerrará. Quedaría como figura a continuación:





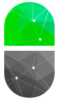
```
7 public class DivisionTryCatch {
8
9     public static void main(String[] args) {
10
11         Scanner scanner = new Scanner(System.in);
12
13         System.out.println("Introduce el dividendo:");
14
15         int dividendo = scanner.nextInt();
16
17         System.out.println("Introduce el divisor:");
18
19         int divisor = scanner.nextInt();
20
21         try {
22
23             int resultado = dividendo / divisor;
24
25             System.out.println("El resultado de la división es: " + resultado);
26
27         } catch (ArithmeticException e) {
28
29             System.out.println("Error: División por cero no permitida.");
30
31         } finally {
32             scanner.close();
33
34             System.out.println("Bloque finally ejecutado. Recursos cerrados.");
35         }
36     }
37 }
```

Bloque finally agregado. Observar como desaparece la advertencia de Eclipse que teníamos antes de agregar el finally (puedes observar la advertencia en el código anterior).

Ejemplo sencillo con instrucción throws: El ejemplo lee un archivo de texto y muestra su contenido. La declaración **throws** se utiliza para indicar que el método **leerArchivo** puede lanzar una excepción **IOException**.

```
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5 import java.io.IOException;
6
7 public class LeerArchivoThrows {
8
9     public static void main(String[] args) {
10
11         try {
12
13             leerArchivo("archivo.txt");
14
15         } catch (IOException e) {
16
17             System.out.println("Error al leer el archivo: " + e.getMessage());
18         }
19     }
20 }
```

Continúa en la siguiente página...

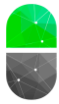


```
21 public static void leerArchivo(String nombreArchivo) throws IOException {
22
23     BufferedReader reader = null;
24
25     try {
26
27         reader = new BufferedReader(new FileReader(nombreArchivo));
28
29         String linea;
30
31         while ((linea = reader.readLine()) != null) {
32
33             System.out.println(linea);
34         }
35     } finally {
36
37         if (reader != null) {
38
39             reader.close();
40
41         }
42     }
43 }
44 }
45 }
46 }
```

En este ejemplo, el método **leerArchivo** declara que puede lanzar una excepción **IOException** utilizando la palabra clave **throws**. Cuando el método **leerArchivo** se llama desde el método **main**, se coloca dentro de un bloque **try-catch** para manejar la excepción **IOException**. Si no lo hiciéramos así, Eclipse marcaría un error en tiempo de compilación (antes de ejecutar el programa) de tipo “**Unhandled Exception**” (Excepción sin capturar), además de sugerirnos como solución incluir el bloque try-catch rodeando la llamada al método.



Obligados a construir un bloque try-catch, cuando ocurra (si es que ocurre una excepción **IOException**, el bloque catch capturaré la excepción y mostrará un mensaje de error.



La declaración **throws** es útil para indicar que un método puede lanzar ciertas excepciones, pero no las manejará dentro del método. En su lugar, delega la responsabilidad de manejar la excepción al código que llama al método, lo que permite un manejo más flexible y modular de las excepciones.

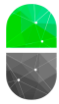
Creación de excepciones personalizadas

En ocasiones, es necesario crear excepciones personalizadas para manejar situaciones específicas en una aplicación. Para crear una excepción personalizada, simplemente se debe extender la clase **Exception** (para excepciones comprobadas) o (para excepciones no comprobadas).

A continuación, tienes un ejemplo en Java que muestra cómo crear y utilizar una excepción personalizada. La excepción personalizada se llama **ExcepcionEdadInvalida** y se lanza cuando se intenta establecer una edad inválida para una persona.

```
3 public class ExcepcionPersonalizada {
4
5     public static void main(String[] args) {
6
7         Persona persona = new Persona();
8
9         try {
10
11             persona.setEdad(-5);
12
13         } catch (ExcepcionEdadInvalida e) {
14
15             System.out.println(e.getMessage());
16
17         }
18     }
19 }
20
21 class Persona {
22
23     private int edad;
24
25     public void setEdad(int edad) throws ExcepcionEdadInvalida {
26
27         if (edad < 0) {
28
29             throw new ExcepcionEdadInvalida("La edad no puede ser negativa.");
30
31         }
32
33         this.edad = edad;
34     }
35 }
36
```

Continúa en la siguiente página...



```
36
37 class ExcepcionEdadInvalida extends Exception {
38
39     public ExcepcionEdadInvalida(String mensaje) {
40
41         super(mensaje);
42     }
43 }
44
45
```

En este ejemplo, creamos una clase **ExcepcionEdadInvalida** que extiende la clase base **Exception**. Esta clase se utiliza para representar una excepción personalizada que se lanza cuando se proporciona una edad inválida para una persona.

La clase *Persona* tiene un método `setEdad`, que lanza la excepción **ExcepcionEdadInvalida** si se proporciona una edad negativa. Cuando se llama al método `setEdad` desde el método `main`, se coloca dentro de un bloque `try-catch` para manejar la excepción **ExcepcionEdadInvalida**. Si se lanza la excepción **ExcepcionEdadInvalida**, el bloque `catch` capturará la excepción y mostrará el mensaje asociado.

La creación de excepciones personalizadas es útil cuando deseas representar y manejar errores específicos de tu dominio de problema, proporcionando mensajes de error más claros y un mejor control sobre el flujo del programa.

Para finalizar, señalar la diferencia entre las instrucciones **“throw”** en singular y **“throws”** en plural, ya que suele ser motivo de confusión:

throw:

La palabra clave **throw** se utiliza para lanzar explícitamente una excepción desde un bloque de código. Se puede utilizar para lanzar excepciones integradas en Java, como `IllegalArgumentException`, o excepciones personalizadas creadas por el desarrollador. Cuando se lanza una excepción con `throw`, el flujo de ejecución del programa se interrumpe y se pasa al bloque `catch` correspondiente, si está presente, o se propaga hacia arriba en la pila de llamadas si no hay ningún bloque `catch` que pueda manejar la excepción.

Ejemplo:

```
public void validarEdad(int edad) {

    if (edad < 0) {

        throw new IllegalArgumentException("La edad no puede ser negativa.");

    } }
}
```



throws:

La palabra clave **throws** se utiliza en la declaración de un método para indicar que el método puede lanzar ciertas excepciones, pero no las manejará dentro del método. En su lugar, delega la responsabilidad de manejar estas excepciones al código que llama al método. Las excepciones enumeradas en la declaración **throws** deben ser capturadas y manejadas por un bloque try-catch en el método que llama, o deben ser declaradas nuevamente en su declaración throws.

Ejemplo:

```
public void leerArchivo(String nombreArchivo) throws IOException {  
  
    // Código para leer un archivo que puede lanzar IOException  
  
}
```

En resumen, **throw** se utiliza para lanzar una excepción explícitamente, mientras que **throws** se utiliza para indicar que un método puede lanzar excepciones pero no las manejará, delegando esa responsabilidad al código que lo llama.